

BSC CIVIL ENGINEERING SCI 3265 COMPUTER SCIENCE II PROJECT. Due 21/2/2025: in Groups of Three.

Python Assignment: Exploratory Data Analysis & Machine Learning

Title: Analyzing and Predicting Wine Quality Using Python

Assignment Objectives

By the end of this assignment, students should be able to:

- ✓ Load and explore datasets using Pandas
- ✓ Perform data cleaning and preprocessing
- ✓ Conduct exploratory data analysis (EDA) using visualization
- ✓ Apply Machine Learning models for prediction
- ✓ Evaluate model performance using classification metrics

Dataset: Wine Quality Dataset

📌 **Download Link:** [UCI Wine Quality Dataset](https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv)

Or use this Python code below to download automatically:

```
import pandas as pd

# Load dataset

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv"

df = pd.read_csv(url, sep=";")

# Display first few rows

df.head()
```

Task 1: Data Exploration

📌 **Instructions:**

1. Load the dataset using **Pandas**
2. Display the first 5 rows
3. Check for missing values and data types
4. Generate summary statistics

Expected code to use

```
import pandas as pd
```

```
# Load dataset
df = pd.read_csv(url, sep=";")

# Check first 5 rows
print(df.head())

# Check for missing values
print(df.isnull().sum())

# Summary statistics
print(df.describe())
```

Task 2: Data Visualization

Instructions:

1. Plot a histogram of **wine quality**
2. Create a correlation heatmap to identify relationships
3. Visualize the distribution of key features like **alcohol** and **pH**

Expected Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Histogram of wine quality
plt.figure(figsize=(6,4))
sns.histplot(df['quality'], bins=6, kde=True, color='blue')
plt.title("Distribution of Wine Quality")
plt.show()
```

```
# Correlation heatmap

plt.figure(figsize=(10,6))

sns.heatmap(df.corr(), annot=True, cmap="coolwarm", linewidths=0.5)

plt.title("Feature Correlation Heatmap")

plt.show()
```

Task 3: Data Preprocessing

Instructions:

1. Convert wine quality into a **binary classification** problem:
 - Quality $\geq 6 \rightarrow$ **Good Wine (1)**
 - Quality $< 6 \rightarrow$ **Bad Wine (0)**
2. Normalize numerical features using **MinMaxScaler**

Expected Code:

```
from sklearn.preprocessing import MinMaxScaler

# Convert quality into binary labels

df['quality'] = df['quality'].apply(lambda x: 1 if x >= 6 else 0)


# Normalize features

scaler = MinMaxScaler()

X = df.drop("quality", axis=1)

y = df["quality"]

X_scaled = scaler.fit_transform(X)
```

Task 4: Train a Machine Learning Model

Instructions:

1. Split the dataset into **train** and **test** sets
2. Train a **Logistic Regression** model
3. Train a **Random Forest Classifier** model
4. Evaluate accuracy, precision, and recall

Expected Code:

```
from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, classification_report


# Split data

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)


# Train Logistic Regression

lr = LogisticRegression()

lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)


# Train Random Forest

rf = RandomForestClassifier(n_estimators=100, random_state=42)

rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)


# Model Evaluation

print("Logistic Regression Performance:")

print(classification_report(y_test, y_pred_lr))


print("Random Forest Performance:")

print(classification_report(y_test, y_pred_rf))
```

Bonus Task: Confusion Matrix **Instructions:**

1. Plot confusion matrices for both models

2. Interpret **False Positives (FP)** and **True Negatives (TN)**

3. Expected Code:

```
from sklearn.metrics import confusion_matrix
```

```
import seaborn as sns
```

```
# Confusion matrix for Logistic Regression
```

```
plt.figure(figsize=(5,4))
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_lr), annot=True, fmt="d", cmap="Blues")
```

```
plt.title("Confusion Matrix - Logistic Regression")
```

```
plt.show()
```

```
# Confusion matrix for Random Forest
```

```
plt.figure(figsize=(5,4))
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_rf), annot=True, fmt="d", cmap="Greens")
```

```
plt.title("Confusion Matrix - Random Forest")
```

```
plt.show()
```

Submission Guidelines

◆ **Deliverables:** A Jupyter Notebook (.ipynb) containing:

✓ **Code Implementation** with explanations

✓ **Plots & Insights from EDA**

✓ **Machine Learning Model Results**

✓ **Confusion Matrix Interpretation**

💡 **Bonus:** Improve model accuracy by experimenting with hyperparameters.???? Extra work optional

Grading Criteria (30 Marks)

Task	Marks
Data Exploration	6 Marks

Task	Marks
Data Visualization	6 Marks
Data Preprocessing	6 Marks
Machine Learning Model Training	6 Marks
Interpretation & Confusion Matrix	6 Marks